

2장 실험

OS 구성과 첫 프로세스 · MIT 6.1810 강의 3과 교재 2장을 xv6에서 직접 돌려 보며

강의 3은 운영체제 설계의 첫 목표를 격리로 잡는다. 사용자 코드는 악의적일 수 있다고 가정하고, 하드웨어 (권한 모드, 가상 메모리)와 신중하게 설계한 시스템 콜로 커널을 지킨다. 이 문서는 그 주장을 xv6에서 확인한 기록이다. gdb를 QEMU에 붙여 전원이 들어온 직후부터 첫 사용자 프로세스가 뜰 때까지 권한 모드와 레지스터가 바뀌는 순간을 직접 보고, 부팅 중 오가는 시스템 콜을 모두 찍고, 사용자 프로그램이 경계를 넘으려 할 때 커널이 무엇을 하는지 시험했다.

2026년 10월 7일 기록. 대상: xv6-riscv 06aad25, 교재 2026-09-23판. 실험은 원본을 건드리지 않도록 복사본(work/xv6)에서 했고, 3절의 시험 프로그램 `priv.c`만 그 복사본에 더했다. 화면 그림은 모두 `tmux` 창에서 실제로 실행한 뒤 `tmux capture-pane`으로 받은 글자를 창 모양으로 그린 것이다. 밑줄 친 쪽 번호를 누르면 교재 PDF의 그 쪽이 열린다.

1. 범위: 강의 3은 교재의 어디인가

강의 3은 교재 2장 Operating system organization (21-30쪽)에 해당한다. 강의 속제는 `kernel/proc.h`, `defs.h`, `entry.S`, `main.c`, `user/init.c`를 읽고 `proc.c`, `exec.c`를 훑어보라고 한다.

강의 3 주제	교재	함께 볼 코드	이 문서
격리, 자원의 추상화	2.1 (22쪽)	-	3절
사용자/감독자 모드, <code>ecall</code>	2.2 (22-23쪽)	<code>user/usys.S</code> , <code>kernel/syscall.c</code>	3절, 7절
모놀리식 커널과 마이크로커널	2.3 (23-24쪽), 그림 2.1	-	4절
xv6 소스 구성	2.4 (25쪽), 그림 2.2	<code>kernel/defs.h</code>	4절
프로세스, 주소 공간	2.5 (26-27쪽), 그림 2.3	<code>kernel/proc.h</code> , <code>proc.c</code>	6절
부팅과 첫 프로세스	2.6 (28쪽)	<code>entry.S</code> , <code>start.c</code> , <code>main.c</code> , <code>init.c</code>	5절
보안 모델	2.7 (28-29쪽)	<code>kernel/trap.c</code>	3절, 8절
실제 운영체제	2.8 Real world (29쪽)	-	-
연습 문제	2.9 Exercises (30쪽)	-	-

2. 실습 환경

1강 실험과 같은 튜체인(`riscv64-unknown-elf-gcc 16.1.0`, `gdb 17.2`)과 `QEMU 11.1.2`를 쓴다. `gdb`로 부팅을 따라갈 때는 CPU를 하나로 줄였다. CPU가 셋이면 세 hart가 모두 같은 중단점에 차례로 걸려 출력이 섞이기 때문이다. 저장소의 `make CPUS=1 qemu-gdb`와 같은 설정이다.

```
# window 1: QEMU waits for gdb (-S) on port 26100
$ qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel \
  -m 128M -smp 1 -nographic ... -S -gdb tcp::26100
# window 2
$ riscv64-unknown-elf-gdb -q -ex 'set architecture riscv:rv64' \
  -ex 'symbol-file kernel/kernel' -ex 'target remote :26100'
```

QEMU는 gdb에 `priv`라는 가상 레지스터를 보여 준다. CPU가 지금 어느 권한 모드에 있는지를 이것으로 바로 읽을 수 있다. 이 문서의 실험은 대부분 이 값을 본다.

3. 격리와 권한 모드

3.1 세 가지 모드

RISC-V CPU에는 권한 모드가 셋 있다. xv6 커널은 S-mode(감독자)에서, 사용자 프로그램은 U-mode에서 돈다. M-mode(기계)는 부팅 직후 잠깐만 쓴다(5절).

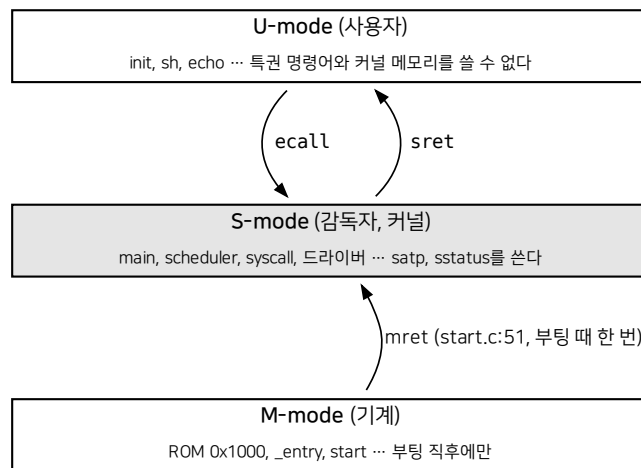


그림 1. RISC-V 권한 모드와 xv6. 사용자 코드는 `ecall`로만 커널에 들어가고, 커널은 `sret`으로 돌려보낸다. M-mode에서 S-mode로 내려가는 `mret`은 부팅 때 한 번 쓰인다.

U-mode에서 막히는 것은 두 종류다. 하나는 **특권 명령어**다. `sstatus`, `satp` 같은 제어 레지스터를 읽고 쓰는 명령어는 S-mode 이상에서만 실행된다. 다른 하나는 **메모리**다. 사용자 프로세스는 자기 페이지 테이블에 있는 주소만 쓸 수 있고, 페이지 테이블은 S-mode에서만 바꿀 수 있다.

강의 노트는 Unix 시스템 콜 인터페이스 자체도 격리를 돕는다고 설명한다. 하드웨어를 직접 넘겨주지 않고 추상화해서 넘겨주기 때문이다.

하드웨어 자원	사용자에게 보이는 추상화	커널이 대신 하는 일
CPU 코어	프로세스 (fork)	코어를 번갈아 나눠 주고, 되돌려 받는다
RAM	주소 공간 (exec, sbrk)	어디에 둘지 정하고, 자기 메모리만 쓰게 한다
디스크 블록	파일	겹치지 않게 관리하고 권한을 지킨다
메모리 공유	파이프	정해진 통로로만 주고받게 한다

3.2 실험: 사용자 프로그램이 선을 넘으면

교재 2.7절은 사용자 코드가 시도할 수 있는 나쁜 일로 “자기 주소 공간 밖 접근, 사용자용이 아닌 명령어 실행, 제어 레지스터 읽기, 시스템 콜에 이상한 값 넘기기”를 든다. 이 중 세 가지를 하는 작은 프로그램 `priv.c`를 복사본에 넣고 실행했다.

```
// priv csr    : read a privileged register in U-mode
asm volatile("csrr %0, sstatus" : "=r"(x));
// priv kmem   : load from a kernel address
volatile uint64 *k = (uint64 *)0x80000000L; ... *k
// priv badsys : ecall with a system call number that does not exist
asm volatile("li a7, 99\n ecall\n mv %0, a0" : "=r"(r) : : "a0", "a7");
```

```
$ priv csr
csrr sstatus in user mode...
usertrap(): unexpected scause 0x2 pid=3
sepc=0x2e stval=0x100025f3
$ priv kmem
load from 0x80000000...
usertrap(): unexpected scause 0xd pid=4
sepc=0x84 stval=0x80000000
$ priv badsys
ecall with a7 = 99...
5 priv: unknown sys call 99
returned -1
still alive
$
```

tmux 캡처. 앞의 둘은 프로세스가 죽고, 셋째는 -1을 받고 계속 산다.

앞의 두 시도는 CPU가 막았다. 둘 다 예외(trap)가 일어나 커널의 `usertrap` (`kernel/trap.c:76--78`)으로 넘어갔고, 커널은 원인을 찍은 뒤 그 프로세스만 죽였다. 쉘과 다른 프로세스는 아무 영향이 없다.

시도	scause	뜻
<code>csrr sstatus</code>	0x2	illegal instruction. <code>stval=0x100025f3</code> 은 그 명령어의 기계어다
<code>0x80000000</code> 읽기	0xd	load page fault. <code>stval</code> 이 읽으려던 주소다
<code>a7 = 99</code> 로 <code>ecall</code>	0x8	정상적인 시스템 콜 진입. 번호 검사에서 걸렸다

셋째는 하드웨어가 아니라 커널 코드가 막았다. `ecall` 자체는 허용된 진입이므로 CPU는 막지 않는다. 대신 `kernel/syscall.c:143`이 번호가 표 안에 있는지 확인하고, 없으면 메시지를 찍은 뒤 반환값으로 -1을 넣는다(`syscall.c:148--149`). 강의 노트가 말하는 두 방어선, 하드웨어의 통제와 신중한 시스템 콜 구현이 각각 드러난 셈이다.

```
num = p->trapframe->a7;
if (num > 0 && num < NELEM(syscalls) && syscalls[num]) {
    p->trapframe->a0 = syscalls[num]();
} else {
    printk("%d %s: unknown sys call %d\n", p->pid, p->name, num);
    p->trapframe->a0 = -1;
}
```

핵심 사용자 코드는 특권 명령어와 커널 메모리를 쓸 수 없다. 거기면 CPU가 예외를 일으키고 커널이 그 프로세스만 끝낸다. 시스템 콜로 들어온 값은 커널이 직접 검사한다.

4. 커널 구성

4.1 모놀리식과 마이크로커널

xv6는 Linux처럼 모놀리식 커널이다. 파일 시스템, 프로세스, 메모리, 드라이버가 모두 S-mode의 프로그램 하나 안에 있고, 시스템 콜 인터페이스가 곧 커널 인터페이스다. 마이크로커널은 커널을 작게 남기고 파일 시스템 같은 기능을 사용자 프로세스(서버)로 뺀다(교재 그림 2.1).

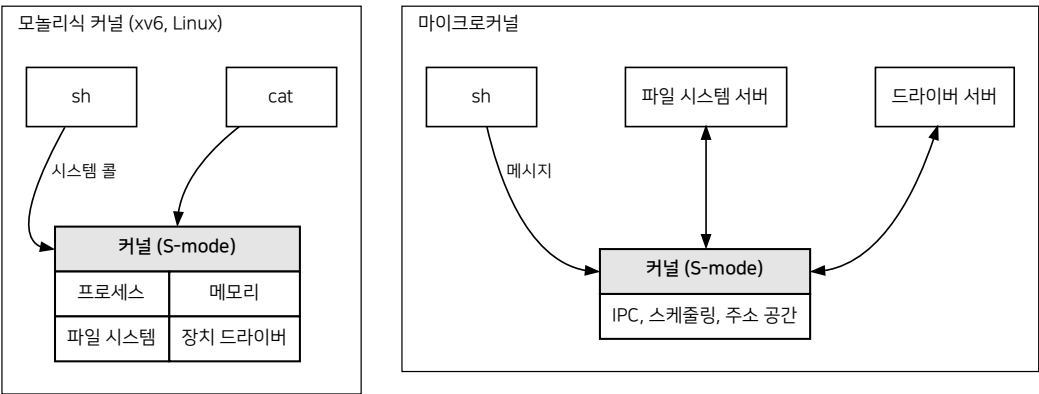


그림 2. 모놀리식 커널과 마이크로커널. 마이크로커널에서 sh가 파일을 읽으려면 커널을 거쳐 파일 시스템 서버에 메시지를 보낸다.

	장점	단점
모놀리식	하위 시스템끼리 바로 협력한다. 예: 파일 시스템과 가상 메모리가 캐시 하나를 같이 쓴다	내부 상호작용이 복잡해 버그가 생기기 쉽다. 커널 안에서는 드라이버끼리도 격리되지 않는다
마이크로커널	커널이 작아 버그가 미치는 범위가 줄어든다	서버 사이에 메시지가 오가 비용이 든다

4.2 xv6 커널 소스 지도

교재 그림 2.2의 파일 목록을 역할별로 묶고 줄 수를 세었다. 커널 전체가 27개 파일, 5500줄이다. 파일 사이의 공개 함수는 kernel/defs.h 한 곳(186줄)에 모여 있어서, 이 파일이 곧 커널 내부 모듈의 목차다.

역할	파일	줄 수
부팅	entry.S start.c main.c	132
프로세스, 스케줄링	proc.c swtch.S	745
트랩, 시스템 콜	trampoline.S kernelvec.S trap.c syscall.c sysproc.c sysfile.c	1230
메모리	kalloc.c vm.c	573
파일 시스템, exec	bio.c log.c fs.c file.c pipe.c exec.c	1653
락	spinlock.c sleeplock.c	171
장치	uart.c console.c printk.c plic.c virtio_disk.c	890
기타	string.c	106
합계	27개	5500

```

host shell - Ubuntu
$ riscv64-unknown-elf-readelf -h kernel/kernel | grep -E 'Class|Machine|Entry'
Class:                ELF64
Machine:              RISC-V
Entry point address:  0x80000000
$ riscv64-unknown-elf-nm kernel/kernel | grep -E ' (_entry|start|main|userinit|fork
ret)$'
0000000080000000 T _entry
0000000080001914 T forkret
0000000080000e60 T main
0000000080000058 T start
0000000080001ba2 T userinit
$ grep -n '0x80000000' kernel/kernel.ld
7:  * ensure that entry.S / _entry is at 0x80000000,
10:  . = 0x80000000;
$ ls kernel/*.c kernel/*.S | wc -l; cat kernel/*.c kernel/*.S | wc -l
27
5500
$ ls -l kernel/kernel fs.img | awk '{print $5, $9}'
2048000 fs.img
281144 kernel/kernel
$

```

tmux 캡처(호스트). 커널 ELF의 진입점은 0x80000000이고 그 자리에 _entry가 있다. 링커 스크립트 kernel.ld:10이 그렇게 배치한다.

핵심 xv6는 모놀리식 커널이다. 커널 전체가 S-mode에서 도는 프로그램 하나이고, 내부 모듈의 경계는 defs.h에 적혀 있다.

5. 부팅: 전원에서 첫 사용자 프로세스까지

교재 2.6절의 흐름을 그림으로 그리면 다음과 같다. 위에서 아래로 갈수록 권한이 낮아진다.

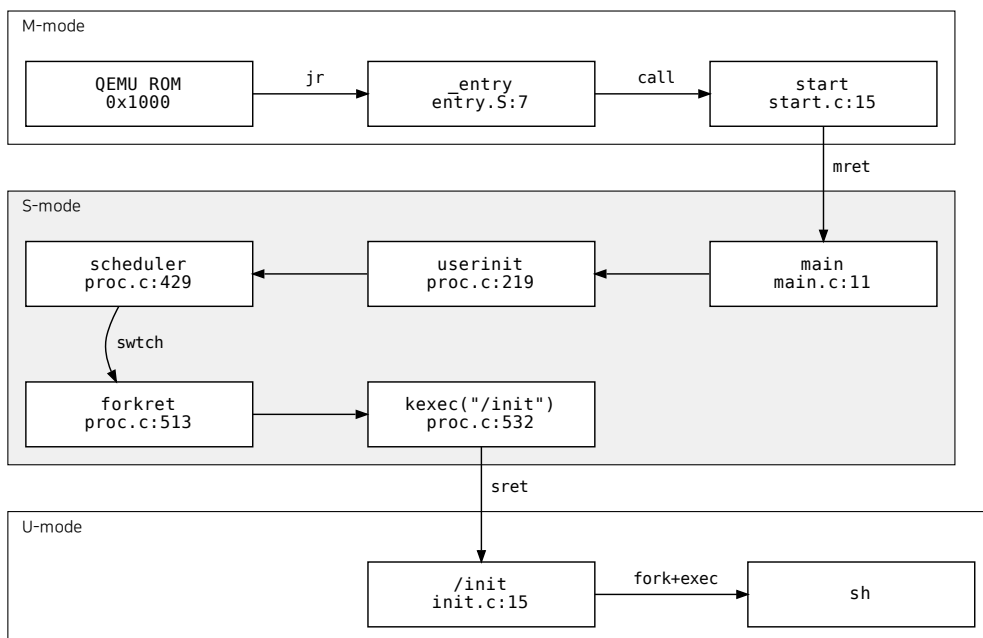


그림 3. xv6의 부팅 경로. 상자 안은 함수와 파일:줄이다. 모드가 바뀌는 곳은 mret과 sret 두 군데다.

5.1 M-mode: ROM, _entry, start

gdb를 붙인 직후 CPU는 0x1000에 멈춰 있다. QEMU가 넣어 둔 작은 ROM 코드이고, 교재가 말하는 “부트로더” 역할을 한다. 여섯 줄이 커널 주소를 읽어 jr로 뛴다.

```
gdb · riscv64-unknown-elf-gdb

(gdb) x/6i $pc
=> 0x1000:      auipc    t0,0x0
0x1004:      addi     a2,t0,40
0x1008:      csrr     a0,mhartid
0x100c:      ld       a1,32(t0)
0x1010:      ld       t0,24(t0)
0x1014:      jr       t0
(gdb) info registers priv
priv          0x3      prv:3 [Machine]
(gdb) b _entry
Breakpoint 1 at 0x80000000: file kernel/entry.S, line 12.
(gdb) c
Continuing.

Breakpoint 1, _entry () at kernel/entry.S:12
12      la sp, stack0
(gdb) info registers pc sp priv
pc          0x80000000      0x80000000 <_entry>
sp          0x0            0x0
priv        0x3            prv:3 [Machine]
(gdb) b start.c:51
Breakpoint 2 at 0x800000ca: file kernel/start.c, line 51.
(gdb) c
Continuing.

Breakpoint 2, start () at kernel/start.c:51
51      asm volatile("mret");
(gdb) p/x $mstatus
$1 = 0xa00000800
(gdb) p/x $mepc
$2 = 0x80000e60
(gdb) info symbol $mepc
main in section .text
(gdb) stepi
main () at kernel/main.c:12
12      {
(gdb) info registers pc priv
pc          0x80000e60      0x80000e60 <main>
priv        0x1            prv:1 [Supervisor]
(gdb)
```

tmux 캡처(gdb). ROM에서 _entry, start를 거쳐 mret 한 명령어로 main에 도착한다. priv가 3(Machine)에서 1(Supervisor)로 바뀐다.

_entry에 도착했을 때 sp는 0이다. C 코드를 부르려면 스택이 있어야 하므로 entry.S:12--17이 stack0에서 hart마다 4096바이트씩 떼어 sp를 잡는다. 그다음 start(start.c:15)가 M-mode에서만 할 수 있는 설정을 한다.

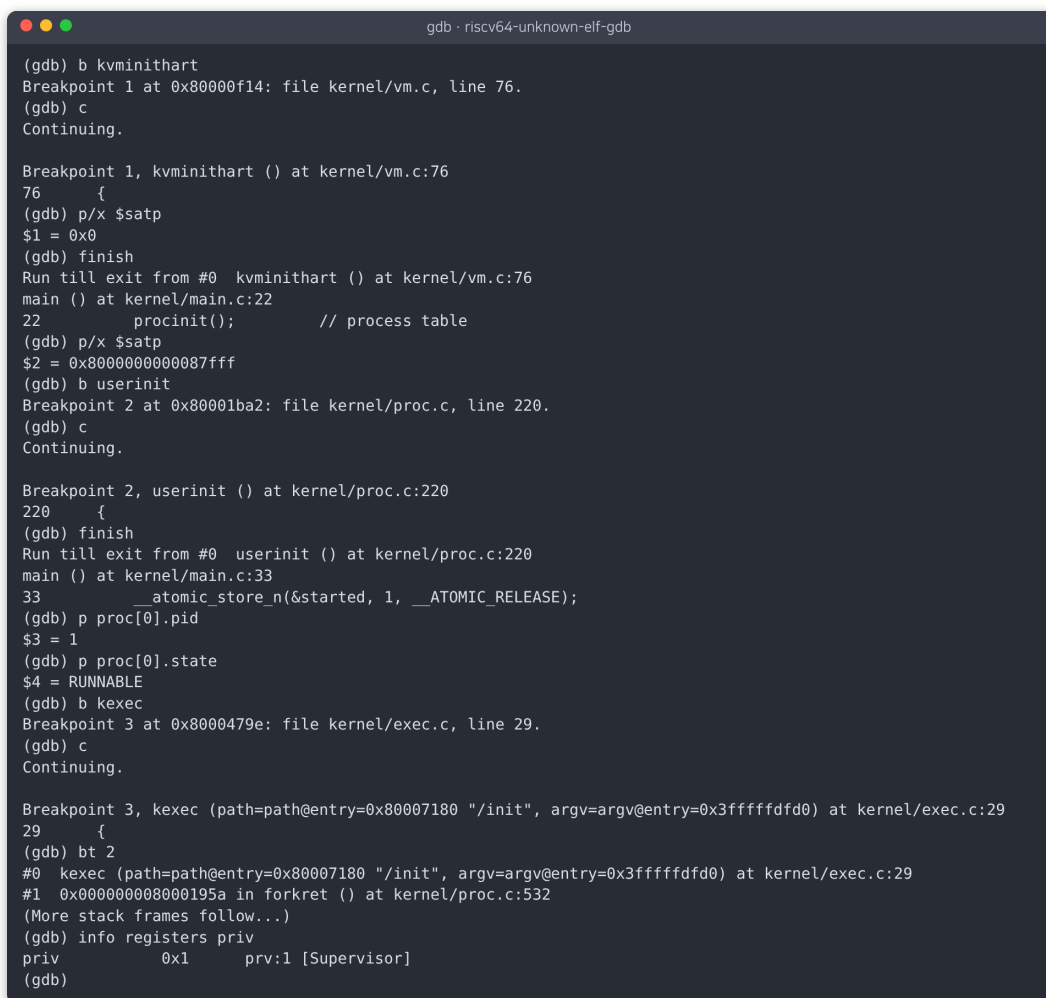
```
x &= ~MSTATUS_MPP_MASK; x |= MSTATUS_MPP_S; w_mstatus(x); // after mret: S-mode
w_mepc((uint64)main); // after mret: pc = main
w_satp(0); // paging off
w_medeleg(0xffff); w_mideleg(0xffff); // traps go to S-mode
timerinit();
asm volatile("mret");
```

gdb로 mret 직전에 읽은 값이 이 코드와 맞는다. mstatus = 0xa00000800에서 11-12번 비트(MPP)가 01, 곧 “돌아갈 모드는 S”이고, mepc = 0x80000e60은 main의 주소다. mret은 이름은 “복귀”지만 실제로는 이 두 값을 써서 권한을 낮추며 점프하는 명령어다. stepi 한 번 뒤 pc는 main, priv는 1이 됐다.

5.2 S-mode: main, userinit, forkret

main(main.c:11)은 CPU 0에서 하위 시스템을 차례로 준비한다.

```
consoleinit(); printkinit(); // console, printk
kinit(); // physical page allocator
kvminithart(); kvminithart(); // kernel page table, turn on paging
procinit(); // process table
trapinit(); trapinithart(); // trap vectors
plicinit(); plicinithart(); // interrupt controller
binit(); iinit(); fileinit(); // buffer cache, inode table, file table
virtio_disk_init(); // disk
userinit(); // first user process
...
scheduler();
```



```
(gdb) b kvminithart
Breakpoint 1 at 0x80000f14: file kernel/vm.c, line 76.
(gdb) c
Continuing.

Breakpoint 1, kvminithart () at kernel/vm.c:76
76      {
(gdb) p/x $satp
$1 = 0x0
(gdb) finish
Run till exit from #0 kvminithart () at kernel/vm.c:76
main () at kernel/main.c:22
22      procinit(); // process table
(gdb) p/x $satp
$2 = 0x80000000000087fff
(gdb) b userinit
Breakpoint 2 at 0x80001ba2: file kernel/proc.c, line 220.
(gdb) c
Continuing.

Breakpoint 2, userinit () at kernel/proc.c:220
220      {
(gdb) finish
Run till exit from #0 userinit () at kernel/proc.c:220
main () at kernel/main.c:33
33      __atomic_store_n(&started, 1, __ATOMIC_RELEASE);
(gdb) p proc[0].pid
$3 = 1
(gdb) p proc[0].state
$4 = RUNNABLE
(gdb) b kexec
Breakpoint 3 at 0x8000479e: file kernel/exec.c, line 29.
(gdb) c
Continuing.

Breakpoint 3, kexec (path=path@entry=0x80007180 "/init", argv=argv@entry=0x3ffffdfdf0) at kernel/exec.c:29
29      {
(gdb) bt 2
#0 kexec (path=path@entry=0x80007180 "/init", argv=argv@entry=0x3ffffdfdf0) at kernel/exec.c:29
#1 0x000000000000195a in forkret () at kernel/proc.c:532
(More stack frames follow...)
(gdb) info registers priv
priv      0x1      prv:1 [Supervisor]
(gdb)
```

tmux 캡처(gdb). kvminithart가 satp를 켜고, userinit이 proc[0]을 만들고, forkret이 kexec("/init")을 부른다.

- kvminithart 전후로 satp가 0에서 0x80000000000087fff가 됐다. 맨 위 4비트 8은 Sv39 페이징 방식, 나머지 0x87fff는 커널 페이지 테이블이 있는 물리 페이지 번호다. 이때부터 커널도 가상 주소로 돈다(3장, 강의 4).
- userinit(proc.c:219) 직후 proc[0]은 pid 1, RUNNABLE이다. 아직 메모리에 프로그램이 없다. 할 일은 allocproc이 정해 둔 context.ra = forkret뿐이다(proc.c:146).

- scheduler가 이 프로세스를 고르면 forkret에서 시작한다. 첫 프로세스일 때만 파일 시스템을 초기화 하고 kexec("/init")을 부른다(proc.c:532). gdb의 bt가 이 호출 관계를 보여 준다.

kexec이 /init을 읽어 주소 공간을 채우고 나면 forkret은 트램폴린의 userret으로 가서 sret으로 U-mode에 들어간다(proc.c:539--542). 여기서부터가 user/init.c다. init은 콘솔을 FD 0, 1, 2로 열고 sh를 fork+exec한다.

핵심 부팅은 권한을 두 번 낮춘다. mret으로 M에서 S로, sret으로 S에서 U로. 첫 프로세스는 커널이 직접 만들고 kexec("/init")으로 채운다.

6. 프로세스

6.1 struct proc

xv6에서 프로세스는 struct proc(kernel/proc.h:82) 하나와 그것이 가리키는 자원이다. 격리에 쓰이는 핵심은 두 가지다. 자기만의 페이지 테이블(주소 공간)과 자기만의 커널 스택이다.

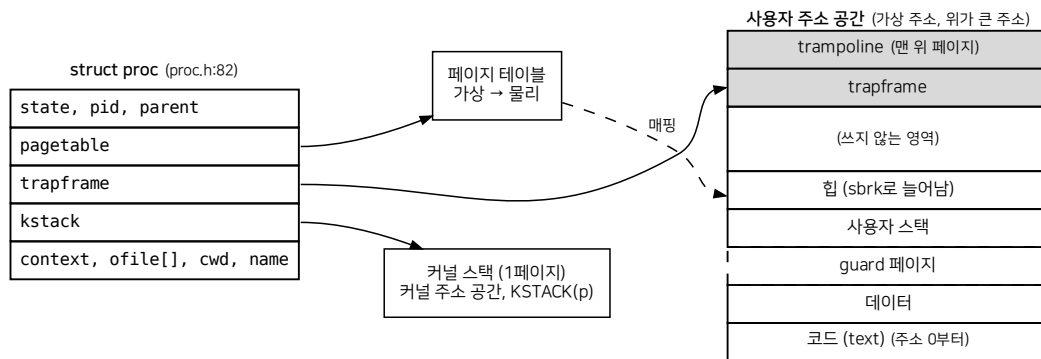


그림 4. struct proc과 주소 공간(교재 그림 2.3). 사용자 주소 공간 맨 위에는 트램폴린과 trapframe 페이지가 있고, 스택 아래에는 접근하면 예외가 나는 guard 페이지가 있다(exec.c:87--95).

- pagetable: 이 프로세스의 가상 주소를 물리 주소로 바꾸는 표. 3절의 0x80000000 읽기가 막힌 것은 이 표에 그 주소가 사용자용으로 없어서다.
- trapframe: 사용자 모드에서 커널로 들어올 때 사용자 레지스터를 저장하는 페이지. 시스템 콜 번호 a7과 반환값 a0도 여기서 읽고 쓴다(7절).
- kstack: 이 프로세스가 커널 안에서 쓰는 스택. 사용자 스택과 따로 있어서 사용자 코드가 망가뜨릴 수 없다.
- state: 아래 그림의 상태 중 하나.

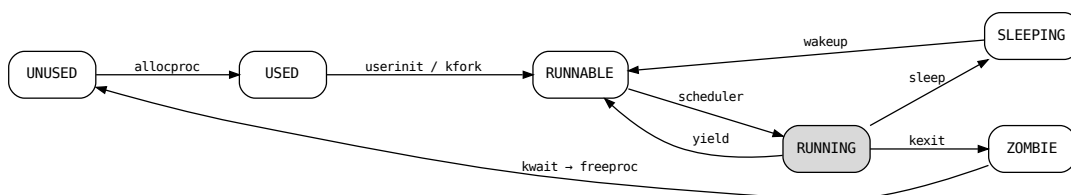
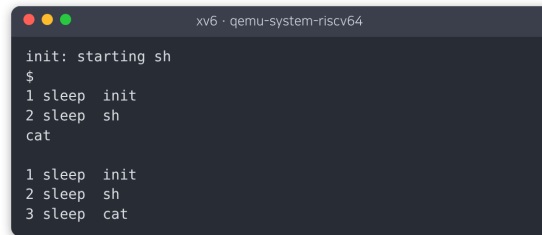


그림 5. enum procstate(proc.h:79)의 전이. 화살표 위는 그 상태로 바꾸는 proc.c의 함수다.

6.2 실험: 지금 있는 프로세스 보기

xv6 콘솔에서 Ctrl-p를 누르면 커널의 `procdump(proc.c:677)`가 프로세스 표를 찍는다. `cat`을 띄워 둔 채 한 번 더 눌렀다.

A screenshot of a terminal window titled 'xv6 - qemu-system-riscv64'. The terminal shows the output of the 'procdump' command. It starts with 'init: starting sh' followed by a prompt '\$'. Then it lists three processes: '1 sleep init', '2 sleep sh', and '3 sleep cat'. The output is repeated twice, suggesting the command was run again.

```
xv6 - qemu-system-riscv64
init: starting sh
$
1 sleep  init
2 sleep  sh
3 sleep  cat

1 sleep  init
2 sleep  sh
3 sleep  cat
```

tmux 캡처. 평소에는 `init(pid 1)`과 `sh(pid 2)`만 있고 둘 다 기다리는 중(sleep)이다. `cat`을 띄우면 `pid 3`이 생기고, 입력을 기다리며 역시 잠든다.

핵심 프로세스는 `struct proc` 하나와 자기 페이지 테이블, 커널 스택, `trapframe`이다. 상태는 6가지이고 `proc.c`의 함수들만 바꾼다.

7. 실험: 부팅 중 오가는 시스템 콜 모두 보기

강의 노트는 `syscall.c`를 고쳐서 시스템 콜을 찍어 보라고 한다. 소스를 고치는 대신 `gdb`의 중단점 명령으로 같은 일을 했다. `syscall.c:143`은 `num = p->trapframe->a7`을 읽은 바로 다음 줄이다. 여기 걸릴 때마다 프로세스 이름, `pid`, 번호를 찍고 멈추지 않고 계속 가게 했다. 부팅이 끝난 뒤 셸에 `echo hi`를 한 번 쳤다.

```

gdb · riscv64-unknown-elf-gdb

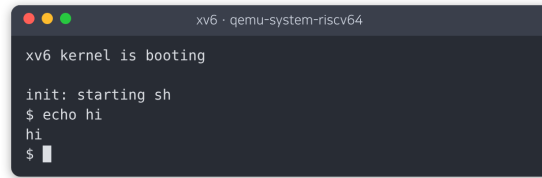
(gdb) break syscall.c:143
Breakpoint 1 at 0x80002880: file kernel/syscall.c, line 143.
(gdb) commands
Type commands for breakpoint(s) 1, one per line.
End with a line saying just "end".
>silent
>printf "%-5s pid=%d a7=%2d\n", p->name, p->pid, num
>continue
>end
(gdb) c
Continuing.
init pid=1 a7=15
init pid=1 a7=17
init pid=1 a7=15
init pid=1 a7=10
init pid=1 a7=10
init pid=1 a7=16
... (a7=16 줄이 이 사이에 16번 더 있다)
init pid=1 a7=16
init pid=1 a7= 1
init pid=1 a7= 3
init pid=2 a7= 7
sh pid=2 a7=15
sh pid=2 a7=21
sh pid=2 a7=16
sh pid=2 a7= 5
sh pid=2 a7= 5
sh pid=2 a7= 5
sh pid=2 a7= 5
sh pid=2 a7= 5
sh pid=2 a7= 5
sh pid=2 a7= 5
sh pid=2 a7= 1
sh pid=2 a7= 3
sh pid=3 a7=12
sh pid=3 a7= 7
echo pid=3 a7=16
echo pid=3 a7= 2
sh pid=2 a7=16
sh pid=2 a7= 5

```

tmux 캡처(gdb). 부팅부터 **echo hi**까지의 시스템 콜 전부. 가운데 같은 줄의 반복만 줄였다.

번호는 `kernel/syscall.h`에 있다. 순서대로 읽으면 5절과 1강에서 본 코드가 그대로 나온다.

누가	번호	무엇 (근거)
init (1)	15, 17, 15	<code>open("console")</code> 실패 → <code>mknod</code> → 다시 <code>open</code> (<code>init.c:19--21</code>)
init (1)	10, 10	<code>dup(0)</code> 두 번으로 FD 1, 2 (<code>init.c:23--24</code>)
init (1)	16×18	"init: starting sh\n" 18글자를 한 글자씩 write
init (1)	1, 3	<code>fork, wait</code>
init (2)	7	자식이 <code>exec("sh")</code> . 아직 이름이 init이다
sh (2)	15, 21	FD 0-2가 열려 있는지 확인하는 <code>open/close</code> (<code>sh.c:152--156</code>)
sh (2)	$16, 5 \times 8$	"\$ " 출력, "echo hi\n" 8바이트를 한 바이트씩 read (<code>ulib.c:gets</code>)
sh (2)	1, 3	<code>fork, wait</code>
sh (3)	12, 7	자식이 명령을 파싱하며 <code>malloc</code> → <code>sbrk</code> , 그리고 <code>exec("echo")</code>
echo (3)	16, 16, 2	"hi", "\n"을 따로 write하고 exit (<code>echo.c:11, 15</code>)
sh (2)	16, 5	다음 "\$ "를 찍고 입력을 기다린다



같은 실행의 QEMU 창. 위의 시스템 콜들이 만든 출력이다.

trapframe->a7에 번호가 들어 있다는 것은 user/usys.S의 `li a7, SYS_...`가 넣은 값을 커널이 trapframe에서 꺼낸다는 뜻이다. 사용자와 커널이 주고받는 것은 함수 주소가 아니라 번호이고, 번호를 함수로 바꾸는 표 `syscalls[]`는 커널 안에 있다. 3절의 `a7 = 99`가 막힌 곳도 이 표의 경계 검사다.

핵심 시스템 콜은 번호로 요청한다. 커널은 `a7`을 검사한 뒤 `syscalls[]` 표에서 함수를 찾아 부르고, 반환 값을 `a0`에 넣는다.

8. 보안 모델

교재 2.7절의 가정을 정리하면 이렇다. 이 문서의 실험은 모두 이 가정 위에 있다.

- 사용자 코드는 악의적이라고 가정한다. 커널은 사용자 프로세스가 자기 메모리, 범용 레지스터 32개, 그리고 시스템 콜이 허락하는 방식 외에는 아무것도 건드리지 못하게 해야 한다(3절 실험).
- 커널 코드는 믿는다. 커널 개발자는 성실하고 커널 함수를 올바르게 쓴다고 가정한다. 그래서 커널은 신뢰 컴퓨팅 기반(TCB)이고, 커널의 버그 하나가 곧 보안 구멍이 될 수 있다.
- 현실은 더 어렵다. 디스크, CPU 시간, 프로세스 표 칸처럼 커널이 지키는 자원을 일부러 다 써 버리는 공격은 막기 어렵다. `proc[NPROC]` (`NPROC = 64`, `param.h:1`)이 가득 차면 `allocproc`은 0을 돌려준다 (`proc.c:122`). 강의 3 숙제가 바로 이 상황을 묻는다.
- 커널은 스스로 이상한 상태를 발견하면 `panic()`으로 멈춘다. 틀린 상태로 계속 도는 것보다 낫기 때문이다.

9. 정리

1. 운영체제 설계의 첫 목표는 격리다. 사용자 코드는 악의적일 수 있다고 가정한다.
2. 격리는 하드웨어(권한 모드, 페이지 테이블)와 신중한 시스템 콜 구현, 두 겹으로 지킨다.
3. RISC-V에는 M, S, U 세 모드가 있다. xv6 커널은 S-mode, 사용자 프로그램은 U-mode에서 돈다.
4. 사용자 코드가 특권 명령어나 커널 메모리를 쓰면 예외가 나고, 커널은 그 프로세스만 끝낸다.
5. xv6는 모놀리식 커널이다. 커널 전체가 S-mode의 프로그램 하나다.
6. 부팅은 ROM → `_entry` → `start` → (`mret`) → `main` → `userinit` → `forkret` → `kexec("/init")` → (`sret`) → `init` → `sh` 순서다.
7. 프로세스는 `struct proc`과 자기 페이지 테이블, 커널 스택, trapframe이다.
8. 시스템 콜은 `a7`에 담긴 번호로 요청하고, 커널이 번호를 검사해 함수로 바꾼다.